

Project 2 Milestone

Team Members: Hung Nguyen, Alex Vargas

Classes:

- **Library_Sim**
 - This class is supposed to represent a library. It has a name and location. It has shelves which contain books for users to look through and check out.
 - Had 3 fields:
 - String Name
 - String Address
 - ArrayList Shelves
 - Has a single constructor which requires a name and address on creation
 - Has getters for all its fields
 - It can add shelves and also view what kind of shelves we have (shelves are organized by book genre)
- **Shelf**
 - This class is supposed to act as a storage for books. As of now, it is all that populates our library
 - Shelves have 5 fields.
 - String Name
 - String Genre
 - ArrayList ArrayList<Books>
 - int Max Rows and Columns
 - Has a single constructor which requires name, genre, max_rows, and max_cols.
 - isFull() for checking if there is space in the shelf or not
 - findBook() to see if target book is in the shelf
 - addBook() adds a book to the shelf if there is space
 - checkInBook() and checkoutBook() to set book availability
 - shelfBookInfo() to print out the info of all books in the shelf
 - toString() is overridden to print shelf and genre
 - Getters for the fields aside from the ArrayList
- **Book**
 - This simulates a book with all of its relevant information for record keeping. It does not actually have the book's contents stored.
 - 5 Fields:
 - String Title
 - String Author
 - int PublicationYear

- String Genre
 - boolean isAvailable
- Constructor that requires all fields as parameters except isAvailable which is initially set to True
- Overridden toString to print out Title, Author, publication year, and availability
- checkIn() and checkOut() to set book availability
- Getters for all the fields
- User
 - This is a parent class which is going to include Librarian and Patron
 - It just simulates a person that can interact with the library
 - Fields:
 - String Name
 - int ID
 - ArrayList books
 - Constructor that just requires Name
 - checkOut() and checkIn() from the book class are going to be abstracted here
- Librarian
 - Subclass of User
 - Meant to simulate a user that has library system management controls
 - No Additional Fields
 - Constructor is just super()
 - addShelf() to add shelves to the library
 - addBook() to add books to a shelf
 - Both of these functions will be abstracted using the associated classes.
- Patron
 - Subclass of User
 - Represents a user that is checking in and out books from the library
 - Fields:
 - int MaxCheckOutCount
 - double fines
 - Constructor is super() plus a MaxCheckOutCount, fines are set to 0.0 by default.
 - pay() allows patrons to pay fines

How the Classes Interact

We have 2 main objects that are interacting; we have the library and the users. The library has shelves and shelves have books. This chain of has-a relationships. Librarians

and patrons are users. This is because they both are entities that can act upon the library. That forms is-a relationships. The librarian should be able to manipulate the library, its shelves, and its books. In contrast, the patrons solely interact with books by checking them in and out. On top of that, the User class has a dependency on the Book class to check in and out. The Librarian class has a dependency with the library_sim to add shelves and with Shelf to add books.